SE4050 – Deep Learning

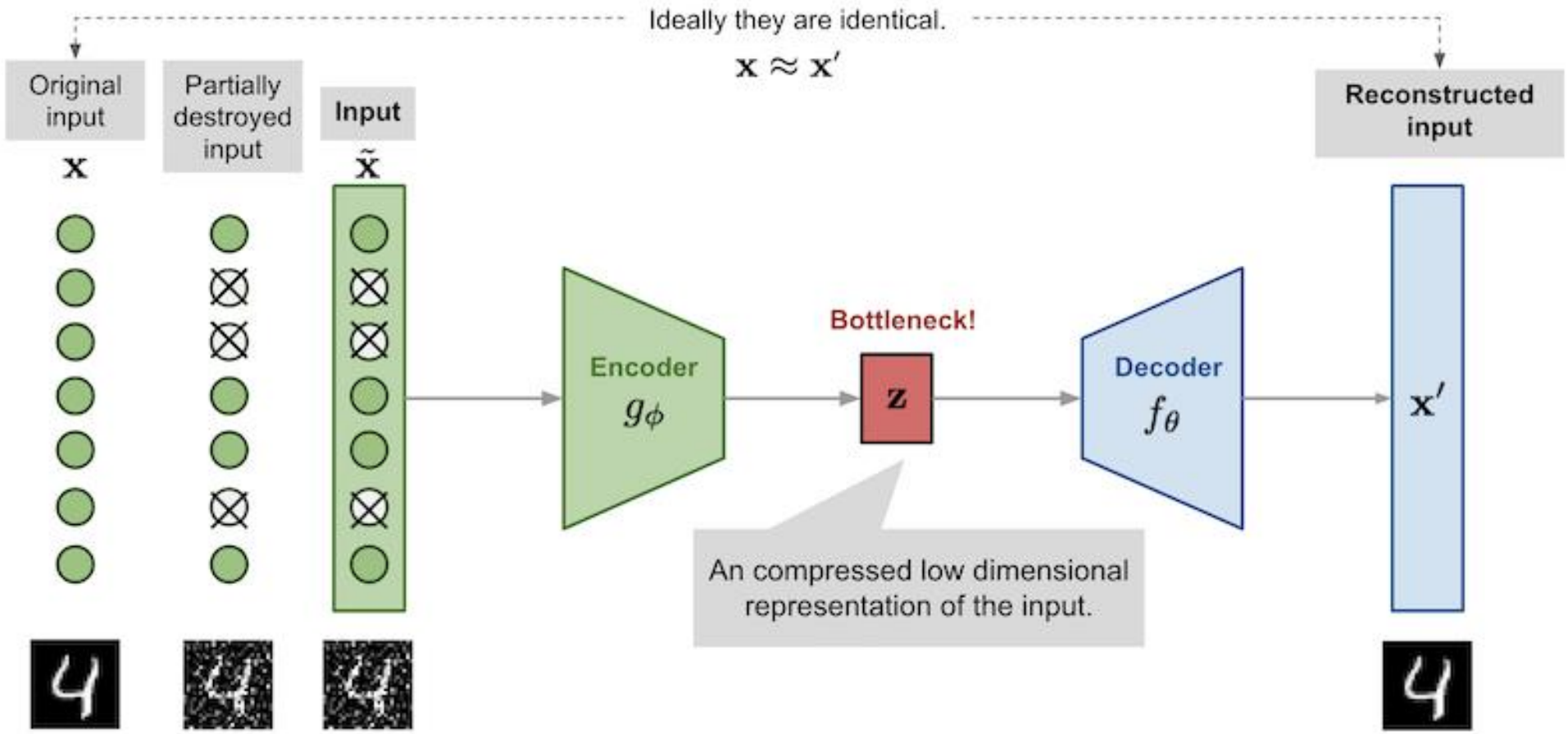
Week 12 – VAEs to GANs

Dr. Mahima Weerasinghe, *MIET*

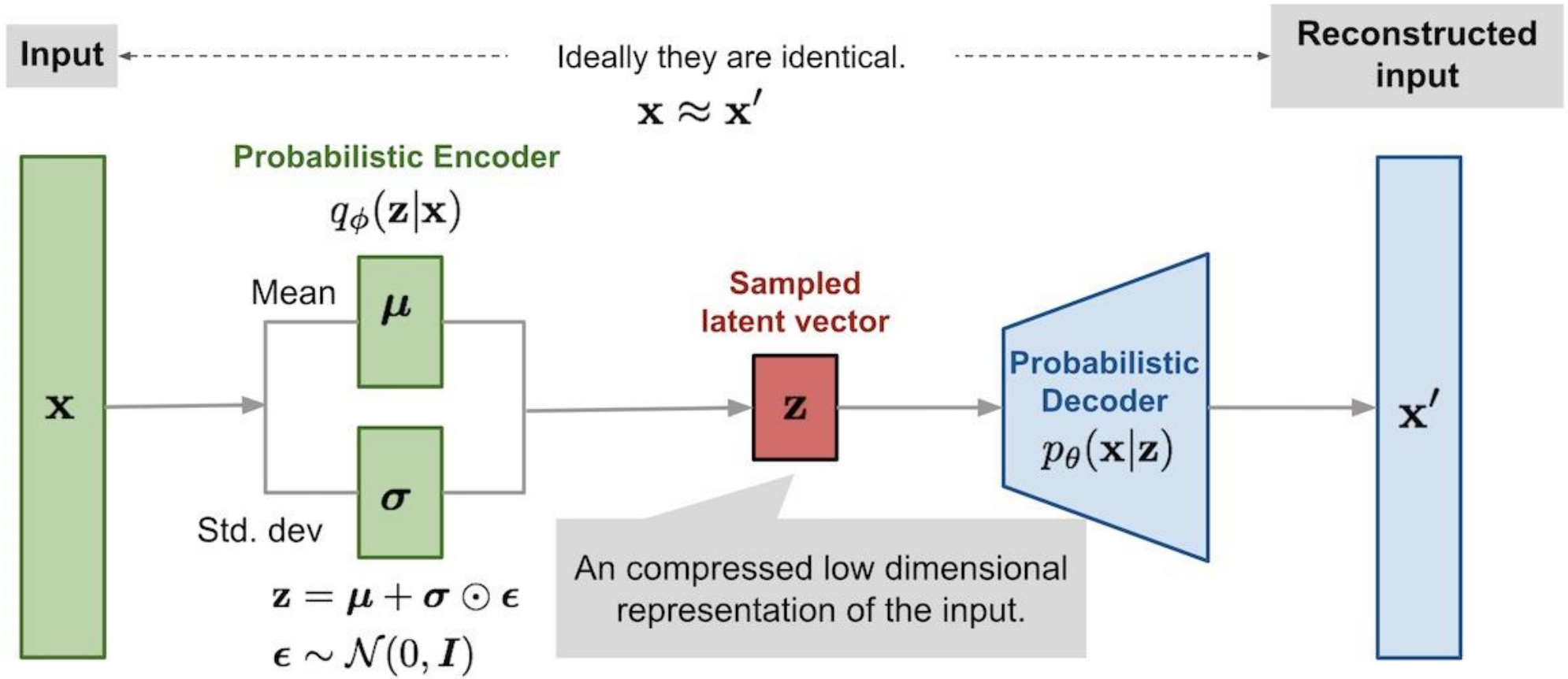
Comparing the utilities of Artificial Neural Networks apart from Classification

1. **Regression**: ANNs can predict continuous outcomes rather than discrete classes. For example, they can model relationships in data to predict prices, temperatures, or other continuous variables.
2. **Generative Modeling**: Some neural networks, like Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs), can generate new data samples that resemble a training dataset. This is different from classification as it involves creating rather than categorizing.
3. **Sequence Prediction**: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are designed to handle sequential data, making them suitable for tasks like time series forecasting, natural language processing (NLP), and speech recognition.
4. **Clustering**: While not a primary function, some neural networks can be adapted for clustering tasks, grouping data points based on similarity without predefined labels.
5. **Feature Extraction**: ANNs can learn to extract relevant features from raw data, which can then be used for various purposes, including classification, but the extraction process itself is not a classification task.
6. **Reinforcement Learning**: In this paradigm, neural networks can be used to learn optimal actions in an environment to maximize cumulative reward, which involves decision-making rather than classification.
7. **Image and Signal Processing**: ANNs can perform tasks such as denoising, inpainting, and super-resolution, which are not classified as traditional classification tasks.

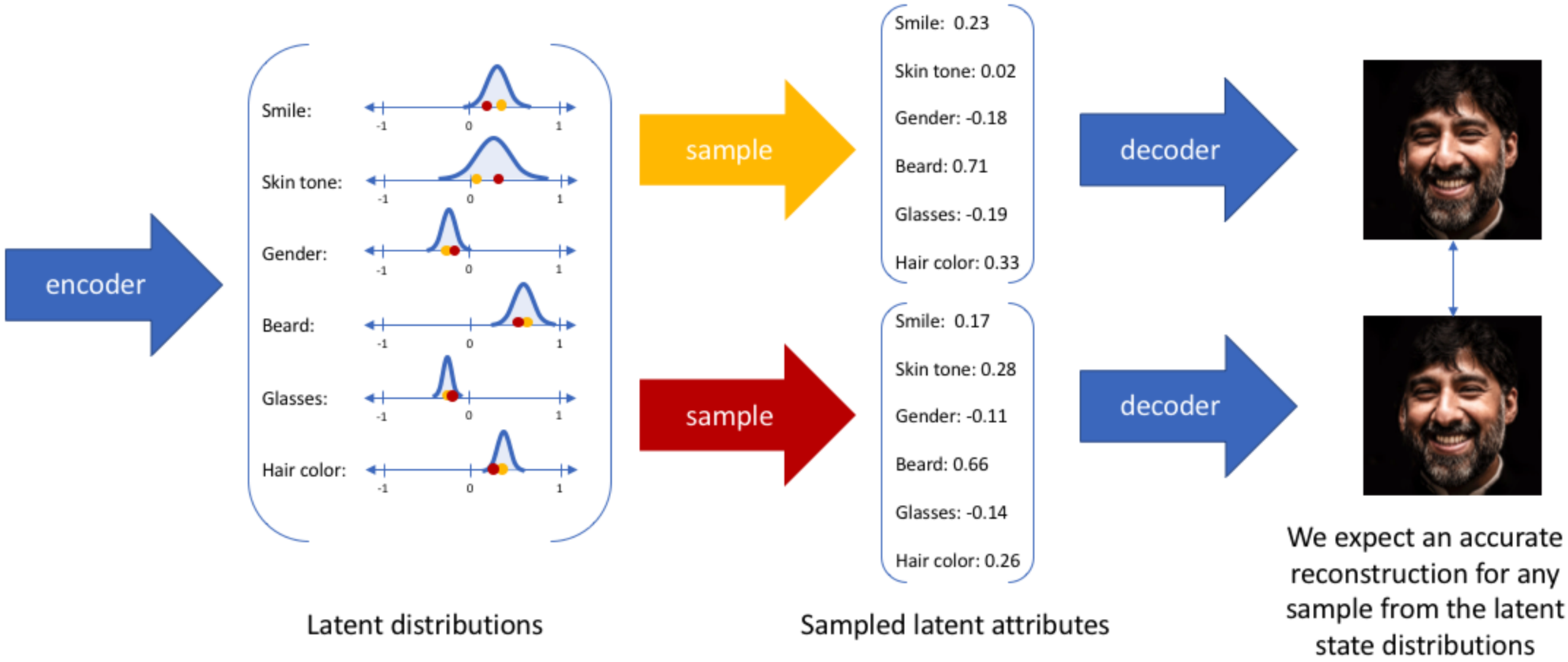
Autoencoders



Variational Autoencoders



Variational Autoencoders

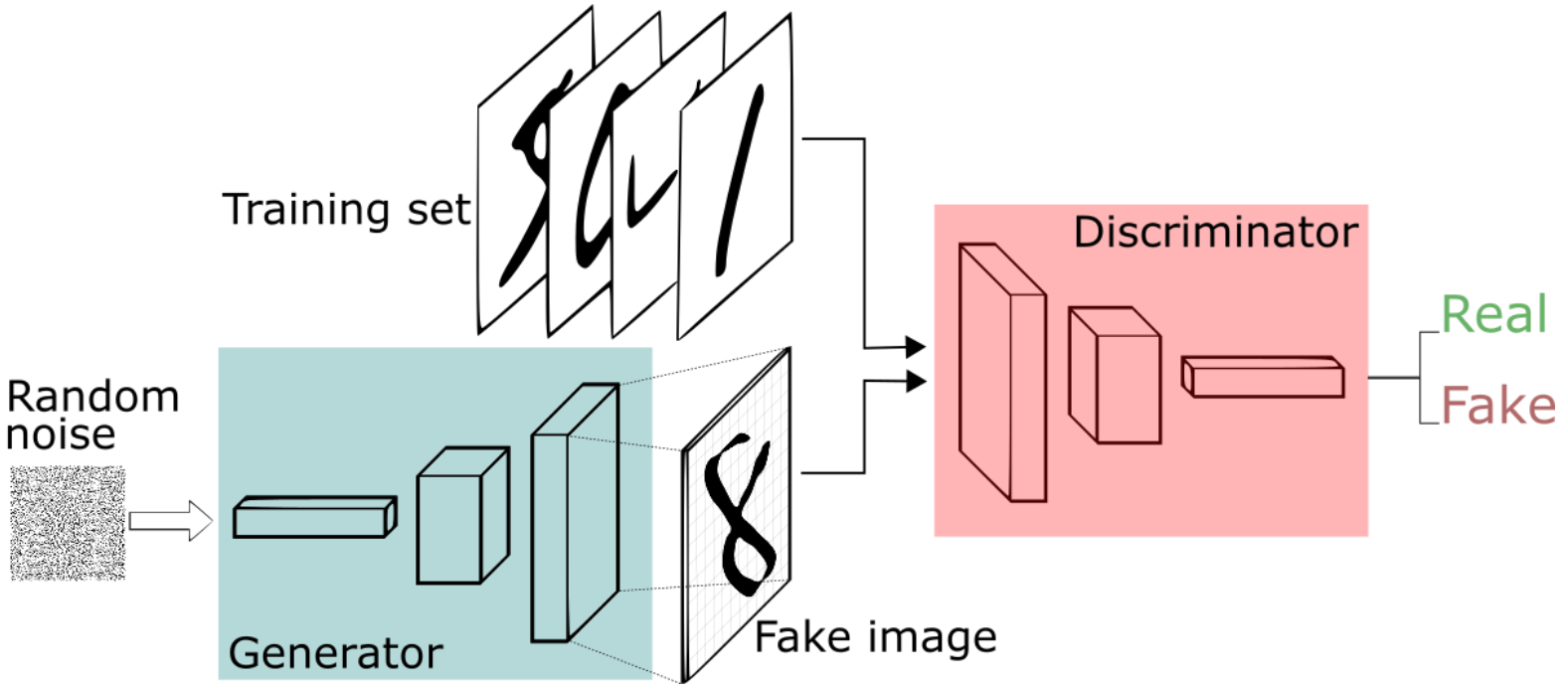


Generative Adversarial Network

A generative adversarial network (GAN) has two parts:

The generator learns to generate plausible data. The generated instances become negative training examples for the discriminator.

The discriminator learns to distinguish the generator's fake data from real data. The discriminator penalizes the generator for producing implausible results.



Utilities of GANs and Challenges

Image Generation and Editing: GANs can create realistic images from scratch, perform image-to-image translation (e.g., turning sketches into photos), and enhance image resolution.

Video Generation and Prediction: They can generate and predict video sequences, which is useful in video synthesis and forecasting.

Data Augmentation: GANs can generate synthetic data to augment training datasets, improving the performance of machine learning models.

Text-to-Image Synthesis: They can convert textual descriptions into corresponding images, aiding in creative industries and design.

Style Transfer: GANs can apply the style of one image to another, useful in art and design.

Medical Imaging: They assist in generating medical images for training purposes, enhancing diagnostic tools.

Protein Engineering: GANs can help in designing new proteins with specific properties.

Astronomical Data Processing: They are used to process and enhance astronomical images.

Training Instability: The adversarial training process can be unstable, leading to difficulties in achieving a balanced and convergent training.

Mode Collapse: GANs sometimes produce limited varieties of outputs, failing to capture the diversity of the real data distribution.

Sensitivity to Hyperparameters: Small changes in hyperparameters can significantly affect the training process and output quality.

High Computational Cost: Training GANs requires substantial computational resources, which can be a barrier for some researchers and organizations.

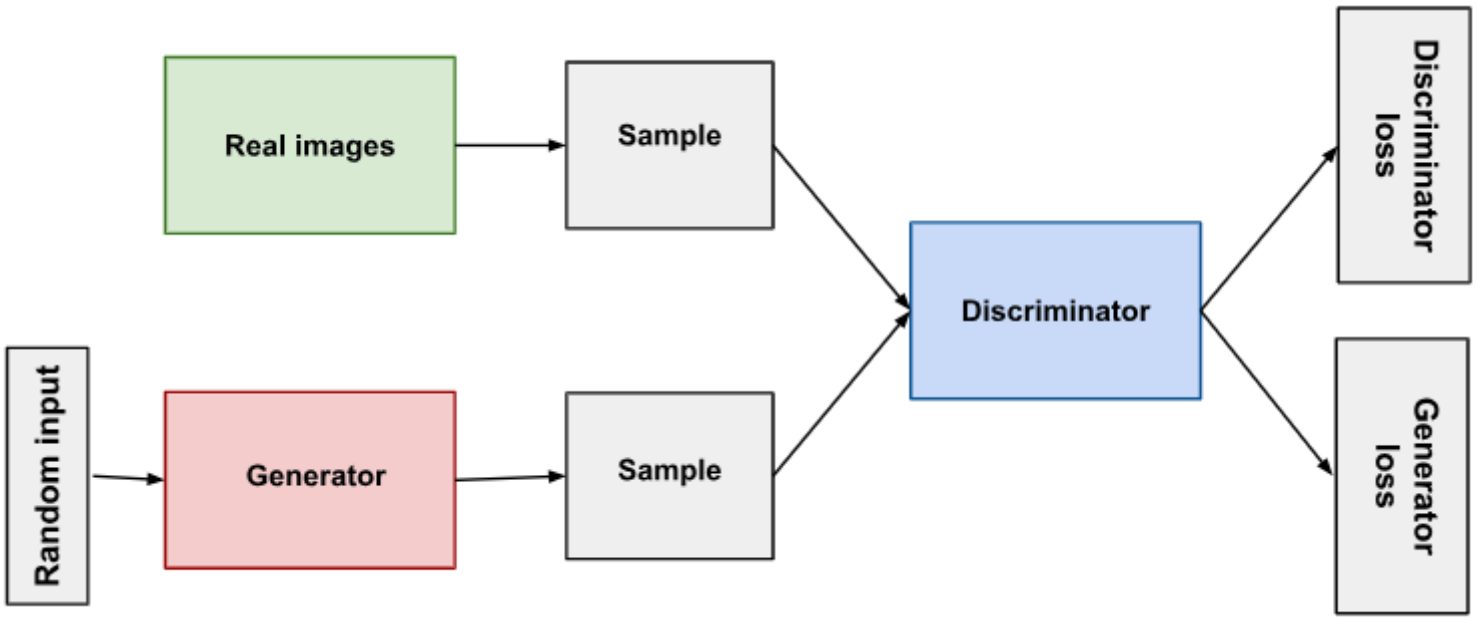
Lack of Evaluation Metrics: Evaluating GAN performance is challenging due to the lack of standardized metrics.

Ethical and Security Concerns: GANs can be used to create realistic fake content, raising concerns about misinformation and privacy.

What is a GAN trying to do ?

As training progresses, the generator gets closer to producing output that can fool the discriminator:

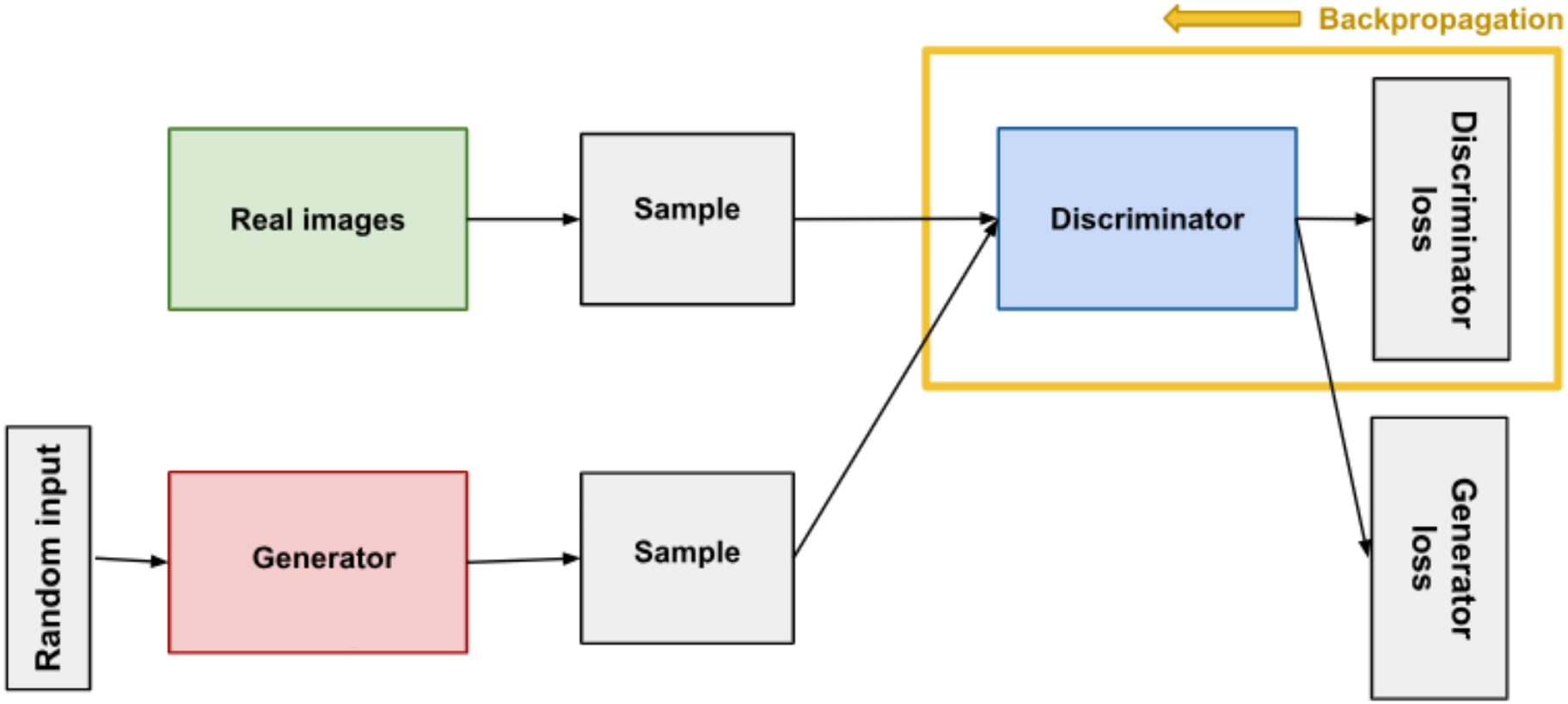
Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.



Both the generator and the discriminator are neural networks. The generator output is connected directly to the discriminator input. Through backpropagation, the discriminator's classification provides a signal that the generator uses to update its weights.

Discriminator

The discriminator in a GAN is simply a classifier. It tries to distinguish real data from the data created by the generator. It could use any network architecture appropriate to the type of data it's classifying.



The discriminator's training data comes from two sources:

Real data instances, such as real pictures of people. The discriminator uses these instances as positive examples during training.

Fake data instances created by the generator. The discriminator uses these instances as negative examples during training.

During discriminator training the generator does not train. Its weights remain constant while it produces examples for the discriminator to train on.

The discriminator connects to two loss functions. During discriminator training, the discriminator ignores the generator loss and just uses the discriminator loss. We use the generator loss during generator training.

During discriminator training:

1. The discriminator classifies both real data and fake data from the generator.
2. The discriminator loss penalizes the discriminator for misclassifying a real instance as fake or a fake instance as real.
3. The discriminator updates its weights through backpropagation from the discriminator loss through the discriminator network.

The Generator

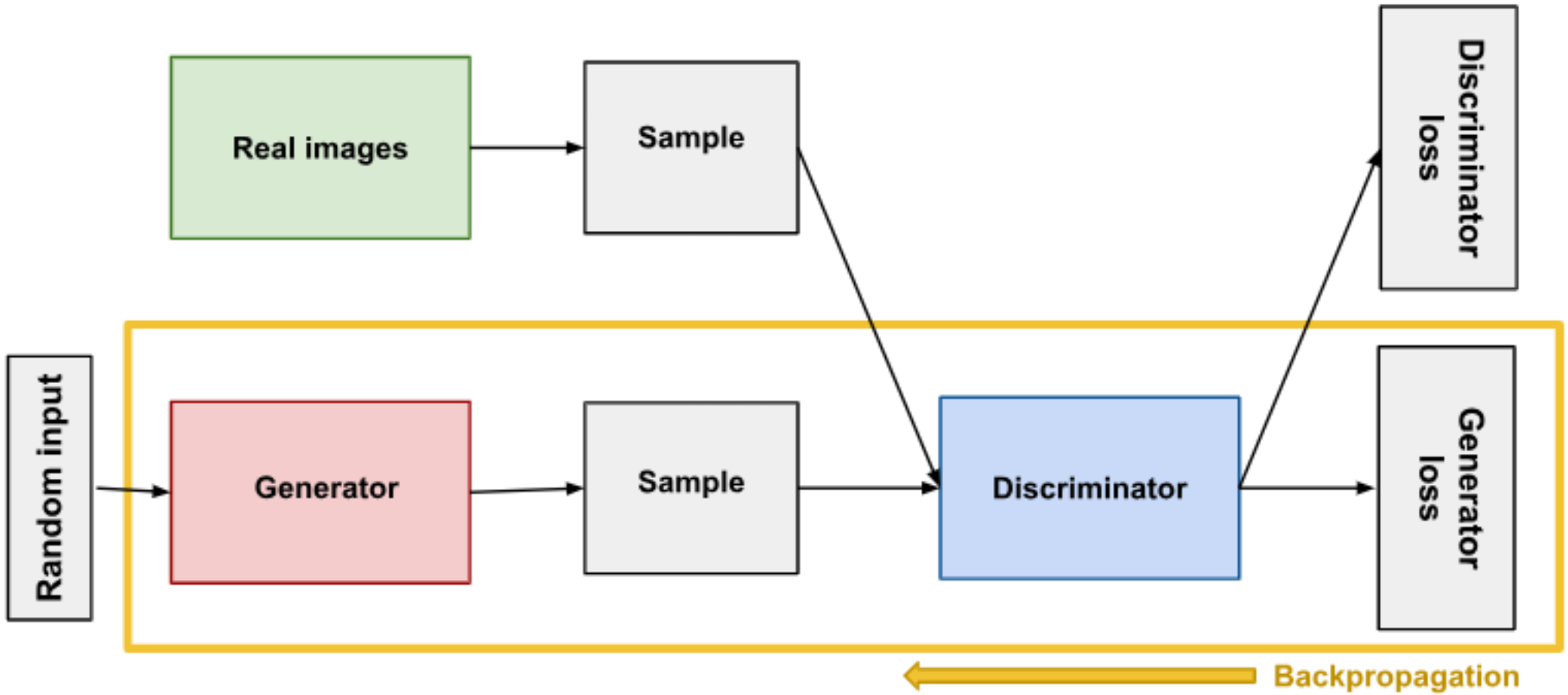
The generator part of a GAN learns to create fake data by incorporating feedback from the discriminator. It learns to make the discriminator classify its output as real.

Generator training requires tighter integration between the generator and the discriminator than discriminator training requires. The portion of the GAN that trains the generator includes:

1. random input
2. generator network, which transforms the random input into a data instance
3. discriminator network, which classifies the generated data discriminator output
4. generator loss, which penalizes the generator for failing to fool the discriminator

Random Input

Neural networks need some form of input. Normally we input data that we want to do something with, like an instance that we want to classify or make a prediction about. But what do we use as input for a network that outputs entirely new data instances?
In its most basic form, a GAN takes random noise as its input. The generator then transforms this noise into a meaningful output. By introducing noise, we can get the GAN to produce a wide variety of data, sampling from different places in the target distribution.
Experiments suggest that the distribution of the noise doesn't matter much, so we can choose something that's easy to sample from, like a uniform distribution. For convenience the space from which the noise is sampled is usually of smaller dimension than the dimensionality of the output space.


Real images
Sample
Discriminator loss
Random input
Generator
Sample
Discriminator
Generator loss
Backpropagation

Using the Discriminator to Train the Generator

In GANs, the generator is not directly connected to the loss that we're trying to affect. The generator feeds into the discriminator net, and the discriminator produces the output we're trying to affect. The generator loss penalizes the generator for producing a sample that the discriminator network classifies as fake.
This extra chunk of network must be included in backpropagation. Backpropagation adjusts each weight in the right direction by calculating the weight's impact on the output — how the output would change if you changed the weight. But the impact of a generator weight depends on the impact of the discriminator weights it feeds into. So backpropagation starts at the output and flows back through the discriminator into the generator.
At the same time, we don't want the discriminator to change during generator training. Trying to hit a moving target would make a hard problem even harder for the generator.

Train the generator with the following procedure:
1.Sample random noise.
2.Produce generator output from sampled random noise.
3.Get discriminator "Real" or "Fake" classification for generator output.
4.Calculate loss from discriminator classification.
5.Backpropagate through both the discriminator and generator to obtain gradients.
6.Use gradients to change only the generator weights.
This is one iteration of generator training. In the next section we'll see how to juggle the training of both the generator and the discriminator.

GAN Training

Because a GAN contains two separately trained networks, its training algorithm must address two complications:

- GANs must juggle two different kinds of training (generator and discriminator).
- GAN convergence is hard to identify.

Alternating Training

The generator and the discriminator have different training processes. So how do we train the GAN as a whole?

GAN training proceeds in alternating periods:

1. The discriminator trains for one or more epochs.
2. The generator trains for one or more epochs.
3. Repeat steps 1 and 2 to continue to train the generator and discriminator networks.

We keep the generator constant during the discriminator training phase. As discriminator training tries to figure out how to distinguish real data from fake, it has to learn how to recognize the generator's flaws. That's a different problem for a thoroughly trained generator than it is for an untrained generator that produces random output.

Similarly, we keep the discriminator constant during the generator training phase. Otherwise the generator would be trying to hit a moving target and might never converge.

It's this back and forth that allows GANs to tackle otherwise intractable generative problems. We get a toehold in the difficult generative problem by starting with a much simpler classification problem. Conversely, if you can't train a classifier to tell the difference between real and generated data even for the initial random generator output, you can't get the GAN training started.

Convergence

As the generator improves with training, the discriminator performance gets worse because the discriminator can't easily tell the difference between real and fake. If the generator succeeds perfectly, then the discriminator has a 50% accuracy. In effect, the discriminator flips a coin to make its prediction.

This progression poses a problem for convergence of the GAN as a whole: the discriminator feedback gets less meaningful over time. If the GAN continues training past the point when the discriminator is giving completely random feedback, then the generator starts to train on junk feedback, and its own quality may collapse.

For a GAN, convergence is often a fleeting, rather than stable, state.

Loss function

GANs try to replicate a probability distribution. They should therefore use loss functions that reflect the distance between the distribution of the data generated by the GAN and the distribution of the real data.

How do you capture the difference between two distributions in GAN loss functions? This question is an area of active research, and many approaches have been proposed. We'll address two common GAN loss functions here, both of which are implemented in TF-GAN:

minimax loss: The loss function used in the paper that introduced GANs.

Wasserstein loss: The default loss function for TF-GAN Estimators. First described in a 2017 paper.

One or Two loss functions?

A GAN can have two loss functions: one for generator training and one for discriminator training. How can two loss functions work together to reflect a distance measure between probability distributions?

In the loss schemes, the generator and discriminator losses derive from a single measure of distance between probability distributions. In both of these schemes, however, the generator can only affect one term in the distance measure: the term that reflects the distribution of the fake data. So during generator training we drop the other term, which reflects the distribution of the real data.

The generator and discriminator losses look different in the end, even though they derive from a single formula.

Minmax Loss

In the paper that introduced GANs, the generator tries to minimize the following function while the discriminator tries to maximize it:

$$E_x[\log(D(x))] + E_z[\log(1-D(G(z)))]$$

In this function:

- $D(x)$ is the discriminator's estimate of the probability that real data instance x is real.
- E_x is the expected value over all real data instances.
- $G(z)$ is the generator's output when given noise z.
- $D(G(z))$ is the discriminator's estimate of the probability that a fake instance is real.
- E_z is the expected value over all random inputs to the generator (in effect, the expected value over all generated fake instances G(z)).
- The formula derives from the cross-entropy between the real and generated distributions.

The generator can't directly affect the $\log(D(x))$ term in the function, so, for the generator, minimizing the loss is equivalent to minimizing $\log(1 - D(G(z)))$.

Other loss functions

Modified Minimax Loss

The original GAN paper notes the minimax loss function can cause the GAN to get stuck in the early stages of GAN training when the discriminator's job is very easy. The paper therefore suggests modifying the generator loss so that the generator tries to maximize $\log D(G(z))$.
In TF-GAN, see modified_generator_loss for an implementation of this modification.